Experiment No 6

Experiment Title: OpenMP Program 4

Problem Statement:

Write a C program utilizing OpenMP directives to demonstrate the behavior of the private clause.

Steps to follow :
Open text editor.
write the below program in it.
Save the file with .c extention.
Compile and execute with given commands.

Objectives

Understand OpenMP Clauses: Learn about the private clause in OpenMP.

Demonstrate Variable Scoping in Parallel Regions: Observe how variable scoping works within OpenMP parallel regions using the private clause.

Implement and Analyze a Parallel Program: Write and execute a program to see the effects of the private clause on a variable in a parallel region.

Theory

OpenMP (Open Multi-Processing) is an API that supports multi-platform shared-memory parallel programming in C, C++, and Fortran. It allows developers to write parallel code using compiler directives, library routines, and environment variables.

Key OpenMP Clause:

private: Specifies that each thread should have its own instance of a variable. Variables are uninitialized at the start of the parallel region.

This clause is used to control the scope and sharing of variables in parallel regions, ensuring that each thread has its own copy of the variable and that changes made to the variable by one thread do not affect others.

Conclusion

This program demonstrates the behavior of the private clause in OpenMP. By using private, each thread gets its own uninitialized copy of the variable val. Changes made to val by each thread are independent

and do not affect the original variable or other threads' copies. This example highlights the importance of variable scoping in parallel programming, ensuring correct and predictable behavior when multiple threads operate concurrently. Understanding how to use the private clause is crucial for managing data correctly in parallel regions, preventing unintended data sharing and race conditions.

Source Code and Output /Screenshots:

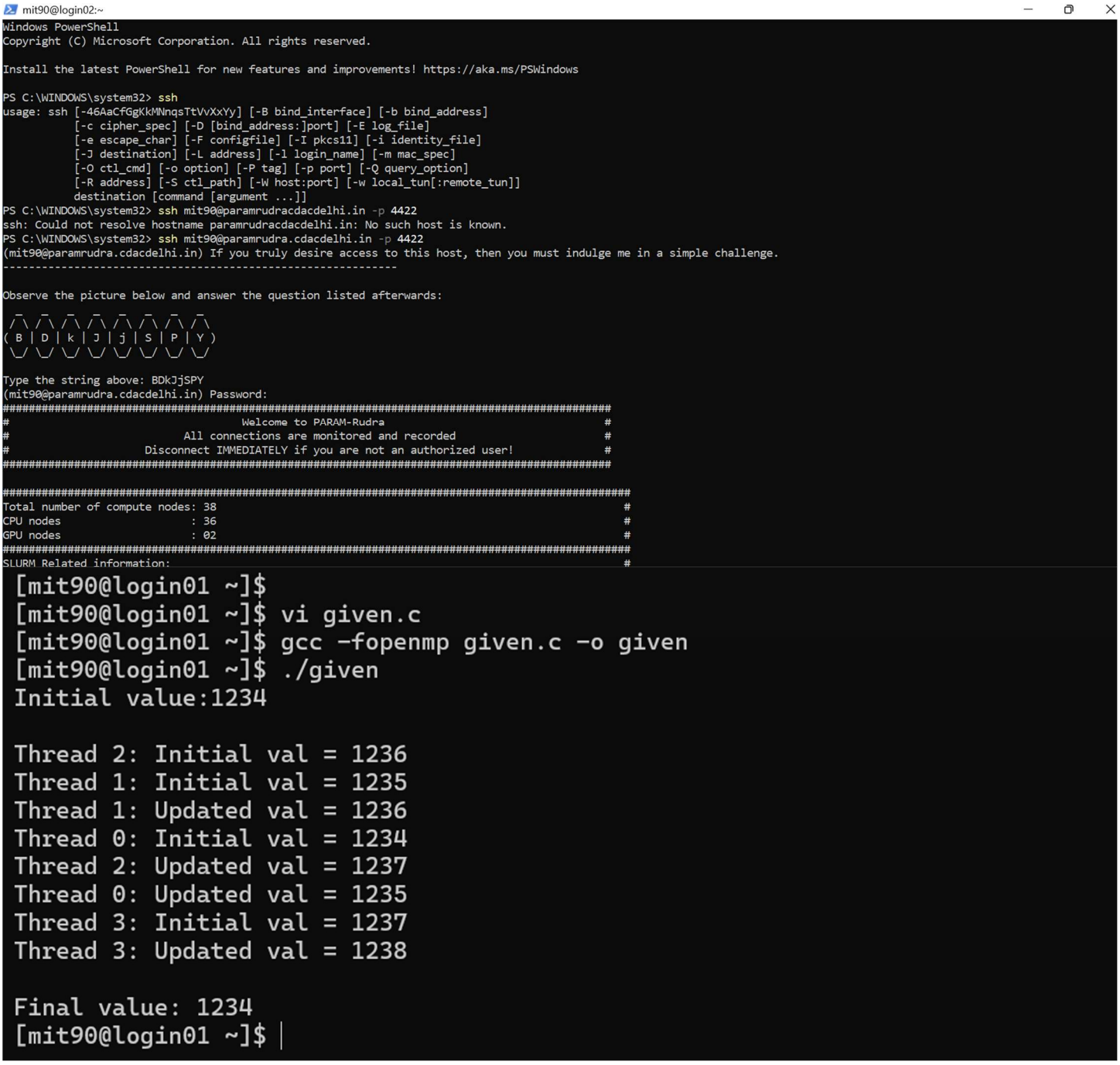
(To be provided by the student)

//Source code and output/screenshot should be available here

mit90@login01:~

```c
#include<stdio.h>
#include<omp.h>
int main()
{
        int val = 1234;
        omp_set_num_threads(4);
        printf("Initial value:%d\n\n",val);

#pragma omp parallel private(val) num_threads(4)
        {
                val =1234+omp_get_thread_num();
                int tid = omp_get_thread_num();
                printf("Thread %d: Initial val = %d\n",tid,val);
                omp_get_thread_num(),val++;
                printf("Thread %d: Updated val = %d\n",tid, val);
        }
        printf("\nFinal value: %d\n", val);
        return 0;
}
~
~
~
```


mit90@login02:~
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.
Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows
PS C:\WINDOWS\system32> ssh
usage: ssh [-46AaCfGgKkMNnqsTtVvXxYy] [-B bind_interface] [-b bind_address]
[-c cipher_spec] [-D [bind_address:]port] [-E log_file]
[-e escape_char] [-F configfile] [-I pkcs11] [-i identity_file]
[-J destination] [-L address] [-l login_name] [-m mac_spec]
[-O ctl_cmd] [-o option] [-P tag] [-p port] [-Q query_option]
[-R address] [-S ctl_path] [-W host:port] [-w local_tun[:remote_tun]]
destination [command [argument ...]]
PS C:\WINDOWS\system32> ssh mit90@paramrudracdacdelhi.in -p 4422
ssh: Could not resolve hostname paramrudracdacdelhi.in: No such host is known.
PS C:\WINDOWS\system32> ssh mit90@paramrudra.cdacdelhi.in -p 4422
(mit90@paramrudra.cdacdelhi.in) If you truly desire access to this host, then you must indulge me in a simple challenge.
Observe the picture below and answer the question listed afterwards:
(B | D | k | J | j | S | P | Y)
Type the string above: BDkJjSPY
(mit90@paramrudra.cdacdelhi.in) Password:
Welcome to PARAM-Rudra
All connections are monitored and recorded
Disconnect IMMEDIATELY if you are not an authorized user!
Total number of compute nodes: 38 #
CPU nodes : 36 #
GPU nodes : 02 #
SLURM Related information: #
[mit90@login01 ~]$
[mit90@login01 ~]$ vi given.c
[mit90@login01 ~]$ gcc -fopenmp given.c -o given
[mit90@login01 ~]$ ./given
Initial value:1234
Thread 2: Initial val = 1236
Thread 1: Initial val = 1235
Thread 1: Updated val = 1236
Thread 0: Initial val = 1234
Thread 2: Updated val = 1237
Thread 0: Updated val = 1235
Thread 3: Initial val = 1237
Thread 3: Updated val = 1238
Final value: 1234
[mit90@login01 ~]$